Polynomial Regression

Contents

1. Overview 2
2. A curved pattern 3
3. Adding powers as new features 4
4. Choosing the degree 6
5. More than one feature 7
6. Summary 10
7. Sources 10
8. Key terms 10

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Multiple linear regression.
- **Leads to:** Bias-variance trade-off; Logistic regression.
- **Compare with:** Overfitting; Kernel trick.

1. Overview

Key point: Polynomial regression fits curves by adding powers of the feature (x^2, x^3, ...) as new features and then running ordinary linear regression on them.

Linear regression fits a straight line (or flat plane). Real relationships are often curved: the output may first fall and then rise, or grow faster and faster. A straight line misses such patterns.

A **feature** (G-772) is an input variable (one column of the data table), an **observation** (G-1374) is one record (one row), and the **target** (G-1949) is the output we predict.

Polynomial regression (G-1515) handles curves without a new algorithm. Polynomial regression creates new features from powers of the existing ones and gives them to plain **linear regression** (G-1094), the way a straight ruler can trace a curve if we feed it bent graph paper. The **degree** (G-577) is the highest power used.

Figure 1 shows the trick on this Note's data. Each point is drawn against x and against a new feature, x^2. In that space the points lie close to a flat plane, and fitting a flat plane is exactly what linear regression does. The black curve is where the data can sit on the plane: seen from the side, it is the U-shaped fit.

with the new feature x², the curve lies on a flat plane:
$\hat{y} = 1.92 + 1.04\ x + 0.82\ x^2$

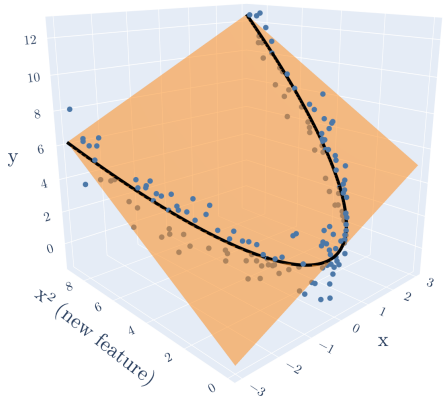


Figure 1: The 160 training points of Section 2 plotted against x and the new feature x^2, with the target upwards. They lie close to the flat plane $\hat{y} = 1.92 + 1.04x + 0.82x^2$, which linear regression fits; the black curve along the plane is the fitted U-shape.

2. A curved pattern

> **Key point:** On data that follows a parabola, a straight line reaches only R^2 0.38.

The example data has 200 points that follow

$$y = 0.8x^2 + 0.9x + 2 + \text{noise}$$

for x between -3 and 3: a U-shaped curve with some random scatter. Fitted with ordinary linear regression, the best straight line scores only $R^2 = 0.38$ on the test points. The R^2 **score** (G-1717) is 1 for a perfect fit and 0 for a model no better than always predicting the mean (see where the number comes from); the **test points** are observations kept out of the fitting. The line cannot bend, so it misses the low middle and both high ends (Figure 2). The misses form a pattern: the points sit above the line at both ends and below it in the middle. A residual pattern like this is the sign of a curved relationship (see assumption 1, a linear relationship).

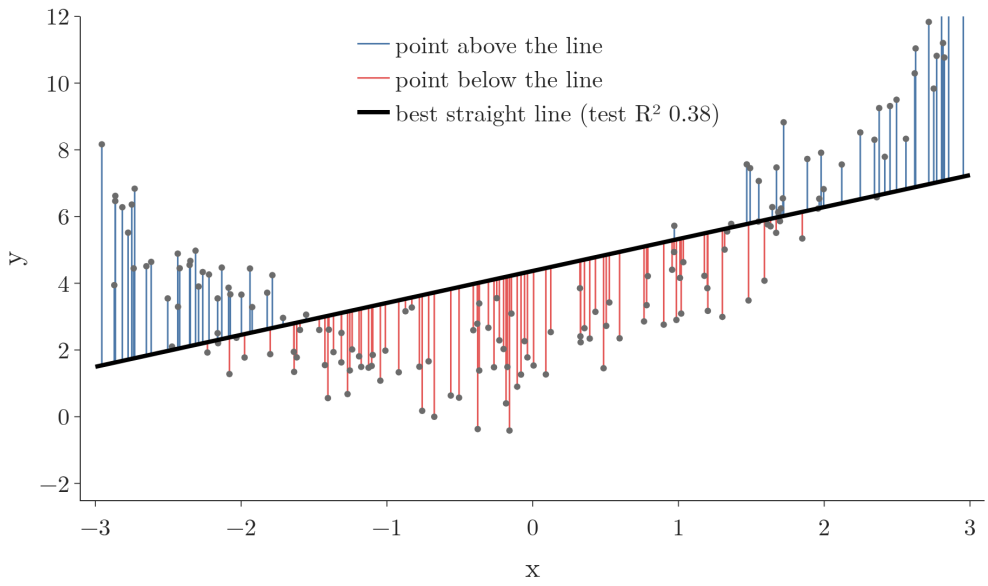


Figure 2: The best straight line on the 160 training points (test R^2 0.38). Blue sticks: points above the line, mostly at both ends. Red sticks: points below it, mostly in the middle.

3. Adding powers as new features

> **Key point:** For degree 2, each observation gets a new feature holding x^2. Linear regression then finds one coefficient per feature.

The trick: for each observation, compute x^2 and add it as a new column of the table (Figure 3).

x
−0.89
1.50
2.10

PolynomialFeatures →

1	x	x^2
1	−0.89	0.79
1	1.50	2.25
1	2.10	4.41

a new column x^2 (and a column of 1s); the model stays linear in its coefficients: $\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2$

Figure 3: `PolynomialFeatures` (G-1516) turns one column into 1, x and x^2

With the features x and x^2, linear regression fits

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2$$

Here $\hat{y}$ ("y hat") is the predicted target and β_0, β_1, β_2 are the coefficients (numbers the fit learns).

The fitted model is a curve in x, but it is still a linear combination of its coefficients: each coefficient just multiplies a feature. This is what "**linear** regression" means here: linear in the coefficients, not in x. With made-up numbers $\beta_0 = 1$, $\beta_1 = 2$, $\beta_2 = 3$ the prediction at $x = 2$ is

$$1 + 2 \times 2 + 3 \times 2^2 = 1 + 4 + 12 = 17$$

and the curve through $x = 0, 1, 2$ gives 1, 6, 17: bending upwards, though every coefficient only multiplies a feature once. So the usual linear regression machinery (OLS, ordinary least squares, or gradient descent) finds β_0, β_1 and β_2 unchanged.

Linearity in the coefficients is why polynomial regression is called "linear": **linear** refers to the coefficients, not to the shape of the curve in x.

> **Python:** Degree-2 polynomial regression.
> ```
> from sklearn.preprocessing import PolynomialFeatures
> from sklearn.linear_model import LinearRegression
>
> poly = PolynomialFeatures(degree=2, include_bias=True)
> X_train_p = poly.fit_transform(X_train)   # [x] -> [1, x, x^2]
> X_test_p = poly.transform(X_test)
>
> lr = LinearRegression().fit(X_train_p, y_train)
> lr.score(X_test_p, y_test)      # R^2 0.83
> lr.coef_, lr.intercept_         # [0, 1.04, 0.82], 1.92
> ```
> `include_bias=True` (G-929) adds the column of 1s; `LinearRegression` fits its own intercept anyway, so that column gets coefficient 0 and either setting works.
> Two things to get right: the powers are made from the features only, never from the target; and `fit_transform` goes on the training features while the test features get the same `transform`, as with any preprocessing step.

With the x^2 feature, the test R^2 jumps from 0.38 to 0.83. The learned equation is:

$$\hat{y} = 1.92 + 1.04x + 0.82x^2$$

It is close to the true equation:

$$y = 2 + 0.9x + 0.8x^2$$

The remaining difference comes from the noise: refitting on the same training observations with the noise removed gives exactly the true equation (notebook).

4. Choosing the degree

> **Key point:** Too low a degree underfits; too high a degree bends to fit the noise and overfits. Training R^2 keeps rising with the degree, but test R^2 falls.

Higher degrees add more features, x^3, x^4 and so on, and the curve can bend more. Figure 4 fits three degrees to only 25 training points, then scores them on 200 new test points.

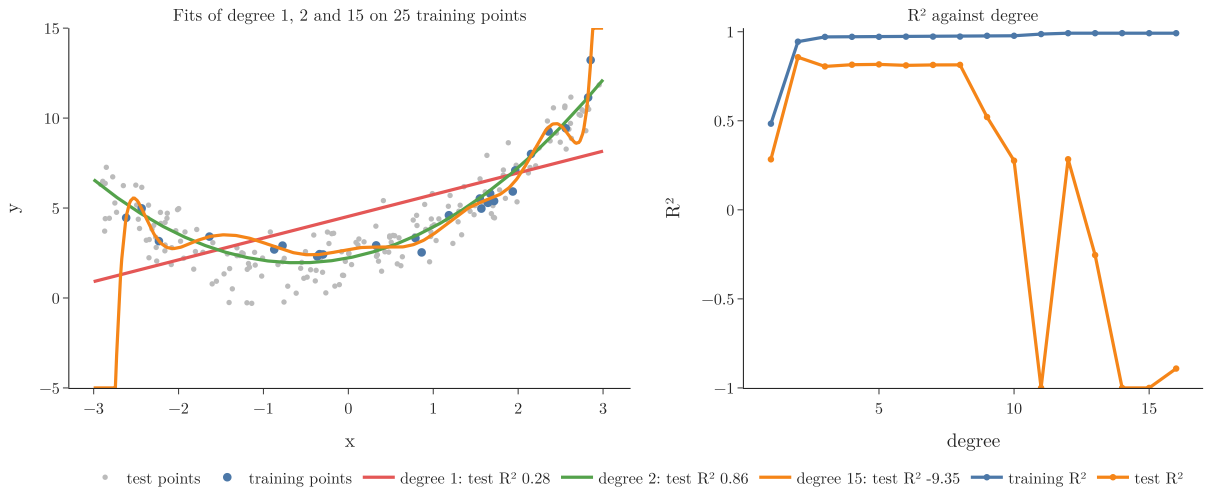


Figure 4: Polynomial fits of degree 1, 2 and 15, and R^2 against the degree

Degree	Training R^2	Test R^2	
1	0.48	0.28	**underfits** (G-2035): too simple to follow the curve
2	0.94	0.86	right shape: matches how the data was made
6	0.97	0.81	starting to fit noise
10	0.98	0.28	overfits
15	0.99	−9.35	overfits badly: wild swings between and beyond the points

The right panel shows the pattern. **Training R^2** rises with every extra degree. The maths guarantees it: a degree $(d+1)$ model contains every degree d curve (set the new coefficient

to 0), so its best training error can only be equal or smaller. **Test R^2** peaks at degree 2 and then collapses: the high-degree curve has learned the noise of these 25 points, not the pattern (this is **overfitting**, G-1429; see variance).

Figure 5 runs through all fifteen degrees one at a time. Watch the curve on the left: up to degree 8 it stays close to the U-shape, then it starts to wiggle between the training points and shoot off at the edges, and the orange test R^2 on the right falls away while the blue training R^2 stays near the top.

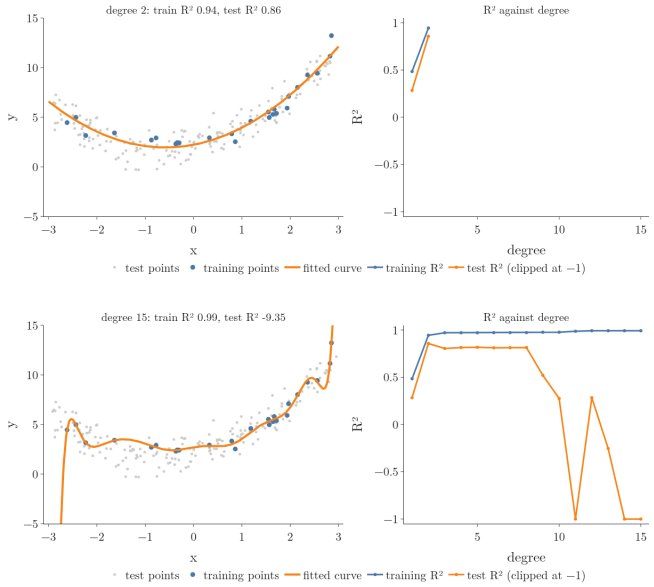


Figure 5: Polynomial fits of degree 1 to 15 on the 25 training points, one degree per frame. Left: the fitted curve against the training points (blue) and 200 test points (grey). Right: training and test R^2 so far; test R^2 is clipped at -1.

The degree is a **hyperparameter** (G-910), chosen by comparing scores on data not used for training, as here, or with cross-validation (a later Note).

> **Extra:** A degree-15 polynomial has terms up to x^{15}; with $x = 3$ that is about 14 million. Columns on such different scales make the fit sensitive to rounding errors. The **condition number** (G-441) measures that sensitivity: the larger it is, the more small rounding errors can change the answer (Goodfellow §4.2). For the 25 training points it is 4.5×10^8 for the raw columns and 1.1×10^6 after `StandardScaler`, so Figure 4 uses a pipeline of `PolynomialFeatures`, `StandardScaler` and `LinearRegression`.

5. More than one feature

> **Key point:** With several features, PolynomialFeatures also adds products of features, such as xy. The number of features grows very fast with the degree.

With two features x and y, degree 2 creates all terms of total power up to 2:

$$1,\ x,\ y,\ x^2,\ xy,\ y^2$$

The product xy is an **interaction term** (G-959): it lets the effect of x depend on the value of y. In maths, take a model with these two terms:

$$\hat{z} = \cdots + \beta_x x + \beta_{xy}\, xy$$

A rise of 1 in x changes $\hat{z}$ by this amount, which depends on y:

$$\beta_x + \beta_{xy}\, y$$

With made-up numbers $\beta_x = 1$ and $\beta_{xy} = 0.5$, at $y = 0$ the change is:

$$1 + 0.5 \times 0 = 1$$

At $y = 4$ the change is:

$$1 + 0.5 \times 4 = 3$$

The fitted model is a curved surface instead of a flat plane.

Figure 6 shows data made from this surface plus noise:

$$z = x^2 + y^2 + 0.2x + 0.2y + 0.1xy + 2$$

Both models are fitted and scored on the same 100 points of Figure 6. A plane reaches R^2 of 0.61; a degree-2 surface reaches 0.98. (The notebook draws its own 100 points and gets 0.38 and 0.98: the plane's score depends on the draw, the surface's barely does.)

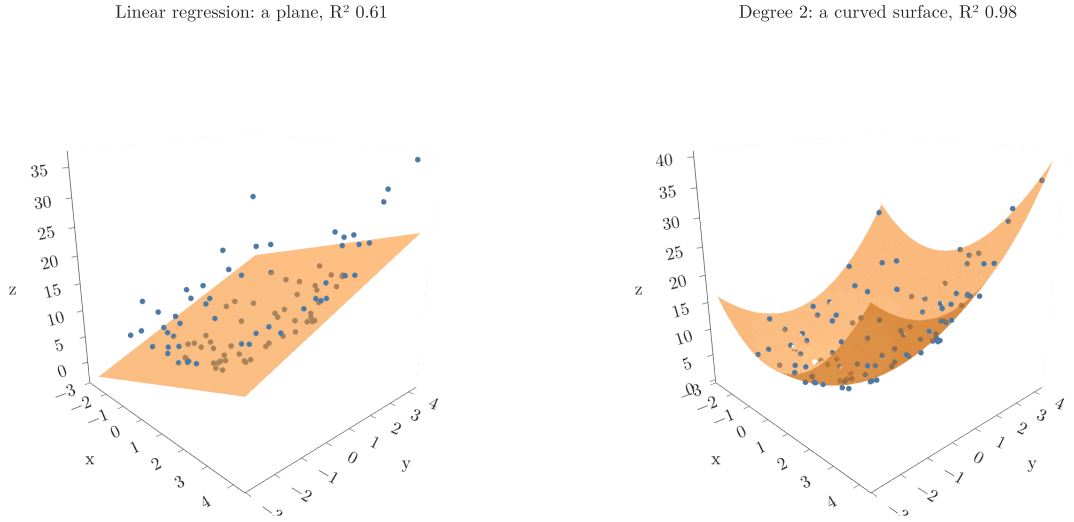


Figure 6: A plane vs a degree-2 surface on curved 3D data

The number of new features grows quickly. With 2 features, degree 2 gives 6; degree 30 gives 496. With many features and a high degree, the new features quickly outnumber the observations. Once there are as many features as observations, least squares passes exactly through every training point, whatever the true relationship, and such a model does very poorly on new data (ISL §6.4.2).

> **Extra:** To put two 1-D arrays side by side as columns, use `np.c_[x, y]` or `np.column_stack([x, y])`. A shortcut sometimes seen, `np.array([x, y]).reshape(100, 2)`, does not do this: it fills the rows with the 100 values of `x` first and then those of `y`, so the first 50 rows each hold two `x` values and the last 50 rows two `y` values. The model still runs, but on scrambled data. Checking `X[:3]` against `x[:3]` and `y[:3]` catches it.

6. Summary

Degree	Features from one original feature	Behaviour on the 25 training points of Section 4	Why
1	x	straight line, underfits (test R^2 0.28)	a line cannot bend to follow the U
2	x, x^2	follows the curve (test R^2 0.86)	it matches how the data was made
15	x to x^{15}	overfits (test R^2 -9.35)	the curve bends enough to learn the noise of 25 points

- Polynomial regression = new power features + ordinary linear regression, so curves need no new algorithm.
- Polynomial regression is "linear" in the coefficients, so OLS and gradient descent work unchanged.
- `PolynomialFeatures(degree=d)` creates the new features; with several features it adds interaction terms too, so the number of features grows very fast with the degree (6 for two features at degree 2, 496 at degree 30).
- The degree controls flexibility: too low underfits, too high overfits. Choose it on held-out data, because training R^2 rises with every extra degree and cannot show overfitting.
- Together these answer the opening question: adding x^2, x^3, ... as new features lets plain linear regression fit curves, and the degree sets how much the curve can bend.

7. Sources

Built from

- CampusX, "Polynomial Regression | Machine Learning", YouTube, https://www.youtube.com/watch?v

Other references

- **Goodfellow**: I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, MIT Press, 2016 (deeplearningbook.org). §4.2 (poor conditioning, condition number).
- **ISL**: James, Witten, Hastie, Tibshirani, *An Introduction to Statistical Learning*, 2nd ed., Springer, 2021. §6.4.2 (what goes wrong in high dimensions).

8. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Polynomial regression (G-1515)	Linear regression on powers (and products) of the inputs, to fit curves.
Degree (G-577)	The highest power used in a polynomial, such as 2 for x^2. It sets the model's complexity: a higher degree lets the curve bend more, and too high a degree overfits.
PolynomialFeatures (G-1516)	scikit-learn transformer that creates the power and product columns.
include_bias (G-929)	PolynomialFeatures setting that adds a column of 1s (the power 0), which acts as an intercept; LinearRegression fits its own intercept anyway, so either setting works.
Interaction term (G-959)	A product of two inputs, such as xy, that lets one input's effect depend on another.
Feature (G-772)	One piece of information about each example that a model uses (e.g. a student's CGPA).
Observation (G-1374)	One record of a dataset: one row of the data table.
Target (G-1949)	The output column we predict, such as the class; also called the label.
Linear regression (G-1094)	A model that predicts a number from the inputs by fitting the straight line (or, with several inputs, the plane) that runs closest to all the training points.
R^2 score (coefficient of determination) (G-1717)	1 minus the model's squared error divided by the squared error of always predicting the mean: 1 is perfect, 0 is no better than the average.
Underfitting (G-2035)	Being too simple to capture the pattern; fails on all data.
Overfitting (G-1429)	Learning the training data too closely, noise included; fails on new data.
Hyperparameter (G-910)	A setting of an algorithm chosen before training, such as a tree's `max_depth`.
Condition number (G-441)	A number that says how much a matrix can magnify small errors, such as rounding errors, when we solve with it; a large value means the answer is sensitive. It is the largest singular value over the smallest, σ_1/σ_n.